

CISCO ENAUTO PREPARATION SERIES

Pi File Cleanup for Git

Lesson 10 - Raspberry Pi, Linux file management, repository preparation, and operational discipline

LAB PLATFORM	PRIMARY SKILLS
Raspberry Pi / Debian 13 / Python 3	find, diff, mkdir, cp, mv, sed, .gitignore
STARTING POINT	END STATE
Loose lesson files, virtual environment, cache, and an old draft	Backed up, sanitized, documented, and organized for Git

MISSION

Prepare the existing ENAUTO Python project for its first Git repository without losing source code, publishing machine-specific clutter, or exposing private lab details.

Core principle: organize, sanitize, and back up before committing.

Learning objectives

- Inventory an existing Linux project with **find**.
- Compare an accidental draft with the current file using **file** and **diff**.
- Create a recoverable backup before changing the working directory.
- Separate executable lessons from written documentation.
- Sanitize public examples and define Git exclusions.
- Verify the physical file structure before initializing Git.

1. Establish the Original State

The project was inspected before anything was moved or removed. This creates a reliable 'before' picture and prevents cleanup by guesswork.

```
cd ~/projects/python
find . -maxdepth 2 -type f -printf '%P\n' | sort
```

Command anatomy

Element	Purpose
find .	Search from the current directory.
-maxdepth 2	Limit the search to two directory levels.
-type f	Return files only, not directories.
-printf '%P\n'	Print paths relative to the starting directory.
sort	Pipe the results into alphabetical order.

Original working files

```
controller.py    lesson3.py      lesson7.py
hello.py         lesson4.py      lesson8.py
lesson1.py       lesson5.py      enauto/
lesson2.py       lesson6.py      __pycache__/
                 lesson6.pyt
```

The inventory revealed three cleanup targets:

- **enauto/** - the local Python virtual environment, which is machine-specific and reproducible.
- **__pycache__/** - generated bytecode created automatically by Python.
- **lesson6.pyt** - an unexpected extension that required inspection before removal.

Operational lesson

Do not delete an unfamiliar file merely because it looks wrong. First identify it, compare it, and preserve a recoverable copy.

2. Compare the Lesson 6 Files

```
file lesson6.pyt
diff -u lesson6.py lesson6.pyt
```

The **file** command identified **lesson6.pyt** as a Python script. The unified diff showed that it was an older hard-coded draft. The current **lesson6.py** contained dynamic hostname, IP discovery, RAM, CPU, and readiness functions.

The old draft also contained an incorrect address entry. It was not suitable for the working project, but it was preserved in the backup so no learning history was lost.

Reading a unified diff

- Lines beginning with - exist in the first file but not the second.
- Lines beginning with + exist in the second file but not the first.
- Unmarked lines provide surrounding context.
- The @@ header identifies the compared line ranges.

3. Create a Safe Backup

The backup was placed outside the future repository. This prevents the repository from accidentally tracking duplicate originals.

```
mkdir -p ~/projects/enauto-backup

cp -p \
  ~/projects/python/hello.py \
  ~/projects/python/lesson1.py \
  ~/projects/python/lesson2.py \
  ~/projects/python/lesson3.py \
  ~/projects/python/lesson4.py \
  ~/projects/python/lesson5.py \
  ~/projects/python/lesson6.py \
  ~/projects/python/lesson6.pyt \
  ~/projects/python/lesson7.py \
  ~/projects/python/lesson8.py \
  ~/projects/python/controller.py \
  ~/projects/enauto-backup/
```

Why these options matter

- **mkdir -p** creates the directory and does not fail if it already exists.
- **cp** copies files while leaving the originals in place.
- **-p** preserves timestamps and file attributes.
- A trailing backslash continues a long shell command onto the next line.

Verify the backup

```
find ~/projects/enauto-backup -maxdepth 1 -type f -printf '%f\n' | sort
```

A backup is not complete until its contents have been verified.

4. Build the Repository Structure

```
cd ~/projects/python
mkdir -p lessons docs
```

The project was divided by purpose:

Path	Responsibility
lessons/	Executable Python lesson files and the controller module.
docs/	Written lesson guides and supporting documentation.
.gitignore	Rules that keep local, generated, and sensitive files out of Git.

Move the lessons

```
mv hello.py lesson1.py lesson2.py lesson3.py lesson4.py lessons/
mv lesson5.py lesson6.py lesson7.py lesson8.py controller.py lessons/
mv lesson6.pyt ~/projects/enauto-backup/lesson6-old-draft.pyt
```

The old draft was moved instead of deleted. This cleaned the project while keeping recovery possible.

5. Sanitize Public Examples

Lessons 2 and 3 contained real local lab values. Before a public GitHub upload, the address and username were replaced with documentation-safe examples.

```
sed -i \  
  -e 's/OLD_IP_ADDRESS/192.0.2.10/g' \  
  -e 's/LOCAL_USERNAME/student/g' \  
  lessons/lesson2.py lessons/lesson3.py
```

Why **192.0.2.10**? It belongs to a range reserved for examples and documentation. It communicates the concept without publishing the live lab address.

Public repository security check

- Never commit passwords, access tokens, private keys, or API credentials.
- Keep **.env** files and private-key formats excluded.
- Replace real internal addresses and usernames when they are not required.
- Do not publish operational, unit-sensitive, or government network information.
- Remember that removing a secret later may not remove it from Git history.

6. Define Git Ignore Rules

A root-level **.gitignore** was created so Git would recognize which local files must never enter source control.

```
# Python-generated files  
__pycache__/  
*.py[cod]  
  
# Python virtual environments  
enauto/  
.venv/  
venv/  
  
# Secrets and local configuration  
.env  
.env.*  
*.pem  
*.key  
  
# Editor and operating-system files  
.vscode/  
.idea/  
.DS_Store  
Thumbs.db
```

Critical distinction

.gitignore does not delete files. The virtual environment and cache remain on the Raspberry Pi. Git will simply leave them out of future snapshots.

Why virtual environments stay local

- They contain machine- and operating-system-specific paths.
- They can be recreated from dependency information.
- They add unnecessary size and noise to a repository.
- Tracking them creates conflicts between different computers.

7. Verify the End State

The project was inspected again after backup, movement, sanitization, and ignore-rule creation.

```
find . -maxdepth 3 -type f -printf '%P\n' | sort
```

Clean project structure

```
.gitignore
docs/
lessons/controller.py
lessons/hello.py
lessons/lesson1.py
lessons/lesson2.py
lessons/lesson3.py
lessons/lesson4.py
lessons/lesson5.py
lessons/lesson6.py
lessons/lesson7.py
lessons/lesson8.py
```

The physical inventory still displayed files under **enauto/** and **__pycache__/**. That was expected: Git had not yet been initialized, and **find** reports physical files regardless of Git ignore rules.

8. Before-and-After Assessment

Before	After
Lesson files mixed in the project root	Python files grouped under lessons/
No documentation directory	Lesson guides grouped under docs/
Virtual environment visible to future Git tracking	Virtual environment protected by .gitignore
Generated Python cache present	Cache excluded from source control
Old draft mixed with active code	Old draft preserved outside the repository
Live lab values embedded in examples	Public examples sanitized

Key takeaway

Git records history; it does not decide what belongs in a professional project. Repository discipline begins before the first commit.

9. Command Reference

Command	Use in this lesson
<code>find</code>	Inventory files and verify the backup and final structure.
<code>sort</code>	Make inventories predictable and readable.
<code>file</code>	Identify the actual type of an unfamiliar file.
<code>diff -u</code>	Compare the current Lesson 6 file with the old draft.
<code>mkdir -p</code>	Create backup, lessons, and documentation directories.
<code>cp -p</code>	Create a source backup while preserving timestamps.
<code>mv</code>	Reorganize active files and relocate the old draft.
<code>sed -i</code>	Replace private lab values inside selected files.
<code>nano</code>	Create and edit the root <code>.gitignore</code> file.

Knowledge check

- Why should a virtual environment stay out of Git?
- What does **-maxdepth** change in a **find** command?
- Why was **lesson6.pyt** compared before it was moved?
- What is the difference between moving a file and ignoring it?
- Why is sanitization required before publishing a repository?
- Why should the first Git commit happen after cleanup?

Lesson completion standard

The lesson is complete when the original source is backed up, active code is organized under **lessons/**, documentation has a dedicated directory, the old draft is outside the project, public examples are sanitized, and the root **.gitignore** defines the local files Git must exclude.

Next lesson

Lesson 11 - Git and GitHub Fundamentals: initialize the repository, verify ignore behavior, review the staging area, create the first commit, connect the remote GitHub repository, and push the project.